1. Let's say, your computer takes 2 seconds to add two numbers. How much time will the following code take to execute? (Assume that assignment operation takes near about 0 seconds)
   ```
   a=5+6
   a=a+10
   ```

2. But, your laptop is slower. It takes 2.5 seconds to add two numbers. Your friend's laptop takes 2 seconds to do so. Which one to take? Just take some constant, c.
3. Let's say, a computer takes c1 seconds to add, c2 seconds to compare, c3 seconds to assign something to a variable. How much time does the following code take?
   ```
   int sum = 0;
   for(int i=1; i<=n; i++)
   {
      sum = sum + i;
   }
   ```

4. Let's say, a computer takes c1 seconds to add, c2 seconds to compare, c3 seconds to assign something to a variable. How much time does it take?
   ```
   int sum = 0;
   for(int i=1; i<=n; i++)
   {
      for(int j=1; j<=n; j++)
      {
            sum = sum + i;
      }
   }
   ```

5. You are searching for an element in an array. Let's say, you are finding element 5. What will happen in these 3 cases? Each row is an array.

7	3	7	30	2	13	9	5
5	4	3	89	23	9	300	8
76	34	4	5	23	87	3	90

n=8 For these 3 cases, how many times will the loop execute?

```
for(int i=1;i<=n;i++)
    if(a[i]==element)
          break;
```

6. Fill up the table. You are finding the Best case, Worst case and Average case steps for different n. Consider c1=c2=c3=1 (Each primitive statement takes 1 second)

n	Best Case	Avg Case	Worst Case
3			
5			
8			
10			
12			
13			
15			

7. Draw the complexity graph.

8.
```
int sum = 0;
for (int i=1; i<=n; i+=2)
{
   sum = sum + i;
}
```

n-> 10
i-> **1, 3, 5, 7, 9** -> 5 items/numbers

i-> 1, 3, 5, 7, 9, 11 -> 6 items/numbers

1, 4, 7, 10,.....(50)
0, 3, 6, 9, ……



1, 3, 5, 7, 9
0, 2, 4, 6, 8

n/2 times ——-> **0.5** * n

G(n) = ……..
= n (coeff) + (coeff)

Time Complexity = O(n)

9. int sum = 0;
for (int i=1; i<=n; **i+=5**)
{
sum = sum + i;
}

n/5 times the for loop will execute
(⅕)***n**

G(n) = ……
= n * (coeff) + (coeff)

Time Complexity = O(n)

10. int sum = 0;
for (int i=n; i>=1; **i-=2**)
{
sum = sum + i;
}

Similarity between prb 8 and prb 10:
for loop will execute n/2 times

Time Complexity = O(n)

11. int sum = 0;
for (int i=n; i>=0; **i-=5**) // (n/5) times
{
sum = sum + i; // O(1) or constant
}

n-> 40
i-> 40, 35, 30, 25, 20, 15, 10, 5, 0

The for loop will (n/5) times……

Time Complexity = O(n)

12. int sum = 0;
    for (int i=1; i<=n; **i+=k**)
    {
       sum = sum + i;
    }
    k is a constant.
    The for loop will execute **n/k** times.
    Time complexity = O(n)

    REFRESH MEMORY FOR LOGARITHMS

    2*2*2*2*2*.....(**10** times) = 2**^**10 = 1024
    Known: 2 and 10
    Unknown: 1024

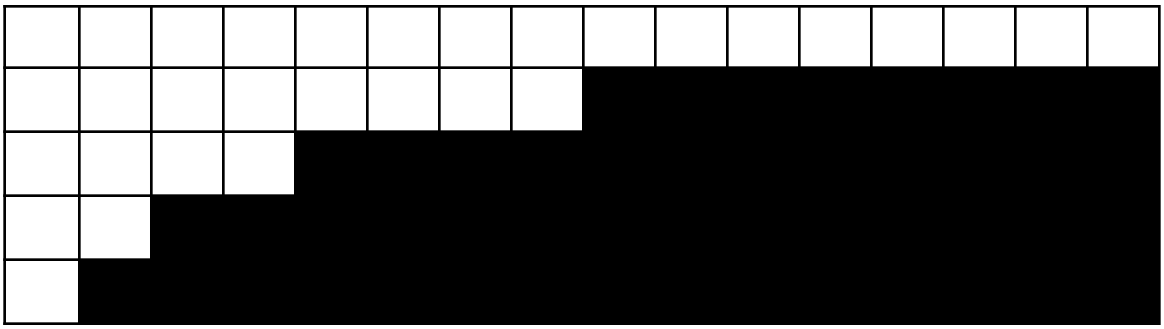
    Logarithm: **How many times** you need to multiply 2s to get 1024?
    Known: 2 and 1024
    Unknown: 10

    log_2 (1024) = 10

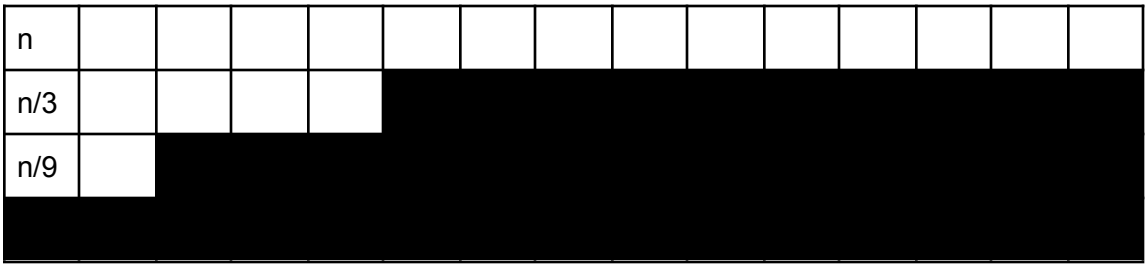
13. int sum = 0;
    for (int i=1; i<=n; **i*=2**)
    {
          sum = sum + i;
    }

    Example-1:
    n=32, i-> 1, 2, 4, 8, 16, 32
           1, (2), (2*2), (2*2*2), (2*2*2*2), (2*2*2*2*2)

    2^**5** = 32
    n=40, i-> 1, 2, 4, 8, 16, 32
    The for loop will execute **log_2(n)** times.



16, divided 2, how many days will you survive? log_2(16) = 4



log_3(15)
log_3(n)

14. int sum = 0;
for (int i=5; i<=n; **i*=2**)
{
sum = sum + i;
}

n = 40, i-> 5, (5*2), (5*2*2), (5*2*2*2)

5, 10, 20, 40
1, 2, 4, 8

log_2(40) = 5.32 -> 5
The for loop will execute **log_2(n)** times.

G(n) = c1 + c1 + c2 * **log_2(n) +** (c1+c5) * (**log_2(n)+1) +** (c4+c1) **log_2(n)**
= **log_2(n) * (coeff) + (coeff)**

Time Complexity = O (log_2(n))
Logarithmic time complexity

15. int sum = 0;
```
for (int i=n; i>=1; i /= 2)
{
   sum = sum + i;
}
```

n= 32, i-> 32, 16, 8, 4, 2, 1
The for loop will execute **log_2(n)** times

16. for(int i=1; i<=n; i*=k)
```
{
    sum = sum + i;
}
```

O (log_k(n))

TRAP

Time complexity will always depend on input size……

Sum = sum + i;
Time complexity = O(1) **that does not depend upon n**

1. **input n**
```
for(int i=1;i<=n;i++) // n times
{
    for(int j=1; j<=100000000; j++)
        sum = sum + i;
}
```

Inner for loop: constant time O(1)
Outer for loop: O(n)
Overall: O(n)

2.
```
for(int i=1;i<=n;i++)
   {
      if(i%2 == 0) break;
      sum = sum + i;
    }
```

Time Complexity = O(1)